Python Project Writeup

Name: Pranav Anand
Reg No: 23MIC0006

Title: Pomodoro Study Timer Website

Motivation

I really like the pomodoro study technique and wanted to make a website similar to pomofocus.io (a website I frequently use). The pomodoro technique works on the principle that a person cannot focus for long periods of time but only short bursts so it splits a one long focus session into multiple shorter ones, each session is followed by breaks to reward the user so they stay motivated. I use this technique for studying for most of my exams and I wanted to make a local hosted version instead of relying on online websites as each of them tend to have some issue- say you reloaded the page or didn't open the site for a long time they stop working or forget to track your focus session.

Event that motivation your project

There were two main events or motivation that made me wanna make a pomodoro study timer for myself->

1. I wanted a local hosted version (so I could use it without internet)
2. Most websites have issues with their session recordings and sometime forget to keep track of your session if not frequently opened.
3. Most websites are just timers and don't have additional features

Architecture

The app has three main parts: a Python server, a SQLite database, and a browser-based interface. Together, they run the timer, save your work, and display the app.

1. The server, `server.py`, starts the web server, prepares the database, and handles requests from the browser. It also serves the app's HTML, CSS, and JavaScript files, checks incoming requests, and connects the study assistant chat to its AI service.
2. The browser interface is built from `index.html`, `style.css`, and `script.js`. It includes the timer, subjects and tasks, statistics, settings, and AI chat. JavaScript manages what appears on the page and uses `fetch()` to request data from the server or send it updates.
3. The SQLite database stores subjects, tasks, notes, completed focus sessions, and timer settings. It also stores the shared timer state, so the server can keep track of the countdown.
4. The timer is managed by the server, not just by the browser. `timer_service.py` keeps track of whether the timer is in focus, short-break, or long-break mode; how much time remains; when the timer should end; the current round; and the timer settings. It also uses a revision number to help prevent an outdated update from one browser tab from replacing a newer one.

5. Since the timer state is stored on the server, the countdown can continue after a page refresh. Keeping that state in one place also helps the timer stay consistent across multiple browser tabs.

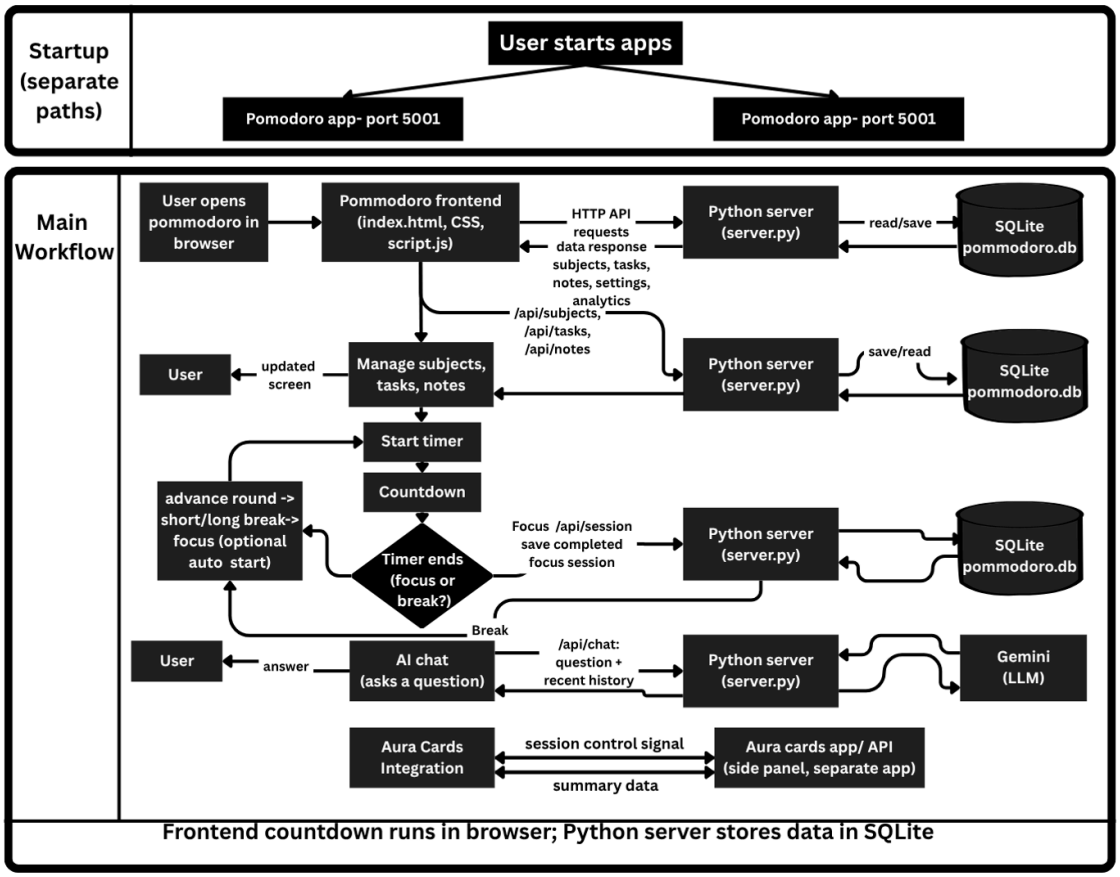


Fig 1: Architecture diagram

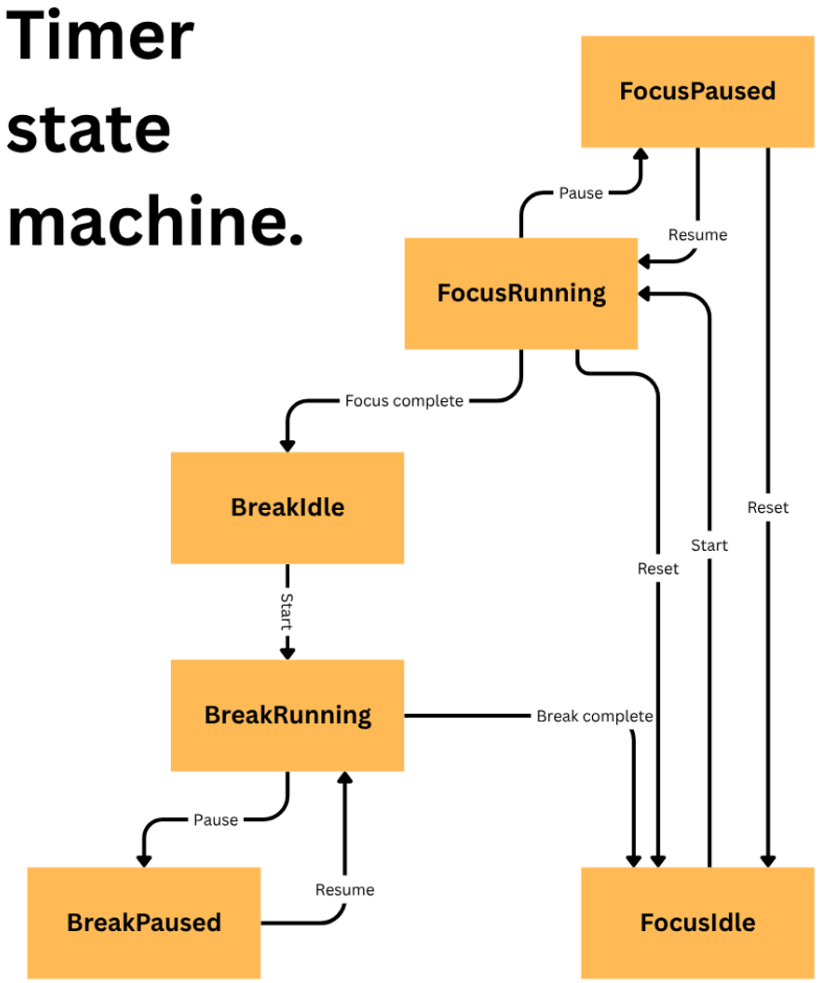


Fig 2: Working of Timer state machine

Complete Techstack info

1. Backend: The app uses Python's built-in `http.server` and `ThreadingHTTPServer`. Its routes and application logic live in `server.py`, with `sqlite3` handling database access. Other built-in modules support tasks such as JSON handling, dates and times, file paths, URL parsing, and secure tokens. It uses a lightweight custom server rather than a framework such as Django, Flask, or FastAPI.

2. Frontend: The interface is built with HTML and CSS, while `script.js` uses vanilla JavaScript to handle the timer, subjects and tasks, statistics, AI chat, and panel interactions. It doesn't use a frontend framework such as React, Vue, or Angular.

3. Database: The app stores its data in `pomodoro.db`, a SQLite database. This includes subjects, tasks, completed sessions, notes, timer settings, and the shared timer state.

4. API: The browser communicates with the backend through custom REST-like endpoints that exchange JSON. For example, it can request app data from `/api/data`, save a session through `/api/session`, update settings at `/api/settings`, or use `/api/chat` for the assistant. These requests are made with JavaScript's `fetch()`.

5. Timer: The countdown is managed using shared server-side state in `timer_service.py`. This keeps the timer authoritative across refreshes and helps prevent an old update from one browser tab from overwriting newer state in another.

6. AI and integrations: The "Study Sidekick" assistant connects to Google Gemini through the `google-genai` Python SDK and uses a system prompt to guide its responses. The app also makes HTTP requests to Aura Cards using configured URLs and tokens to sync summaries. The timer API applies a same-origin check.

7. Configuration and deployment: Environment variables provide settings such as the host, port, AI keys, and integration tokens. The project also includes `render.yaml` and runtime settings suited to deployment on Render.

In short, the Pomodoro app is a Python-and-SQLite web app with a custom server and a plain HTML, CSS, and JavaScript frontend—not a framework-based app.

Flow of info b/w files in Pomodoro.

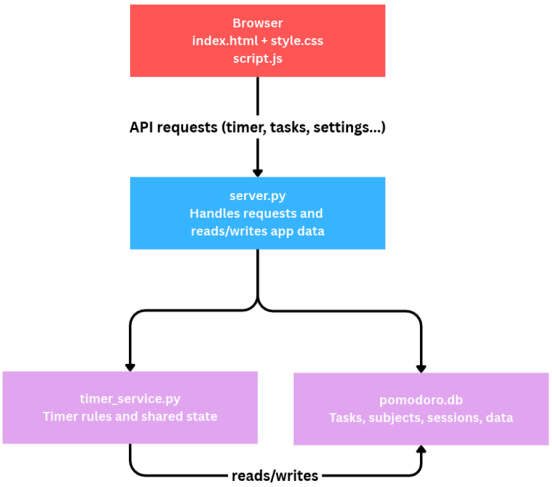


Fig 3: Flow of information between files in Pomodoro

Datasets used

Doesn't use any datasets

Prompts used

1. Auracards-main index.html requirements.txt server.py style.css
   README.md pomodoro.db script.js study_data.json

   this is the file breakdown rn all the files u see are mine
   all his files are inside Auracards-main

   sorry it's in

   /mnt/c/Users/prana/Pomodoro-Website-Python-Project/Auracards-main/Auracards-main
   $ ls
   CHANGELOG.md app.py dist examples start.bat tests
   README.md backend docs requirements.txt tools
   ok now gimme a prompt so i can give VScode and it does the whole thing I guess

2. how do u export the API on powershell
3. no i mean the gemini api
4. how do i go about hosting on render also is it free?
5. ok there's a few issues with this I think- first the gemini API key
   second the project is run from windows using
   .\start_both.bat

   and i guess that file has some info in it

6. what's my start command?
7. How do u add API key like
   GEMINI_API_KEY in power shell terminal again

API Integration

1. Two separate apps: Pomodoro and Aura Cards run as independent Python applications. They communicate over HTTP using configured service URLs and shared tokens, rather than being combined into one large application.

2. Configuration tells the apps where to connect. Environment variables such as `AURACARDS_URL`, `POMODORO_URL`, and `STUDY_INTEGRATION_TOKEN` provide service addresses and authentication details. The Pomodoro server uses these settings to build requests to Aura Cards and other integration endpoints.

3. A local helper connects the backends. The `peer_call()` function in `local_bridge.py` sends JSON requests between the apps. It restricts these direct calls to loopback addresses such as `localhost` and `127.0.0.1`. For example, a request from the browser to Pomodoro may cause Pomodoro's backend to ask Aura Cards for deck data, then return Aura's response to the browser.

4. Both apps can use the same timer. The timer's authoritative state is stored in SQLite by `timer_service.py`, rather than only in the browser. It tracks details such as the current mode, remaining time, deadline, revision number, and any linked task or deck.

The frontend can send actions such as start, pause, reset, change mode, or select a task to `POST /api/timer`. The server checks that the request is same-origin, validates the revision to guard against outdated updates from another tab, and updates the shared timer state. This lets both the Pomodoro interface and Aura companion work with the same timer.

5. Summary data is exchanged through a protected endpoint. Pomodoro's `GET /api/integrations/summary` endpoint requires a bearer token. It returns aggregate progress, such as today's and all-time session and minute totals—not the app's full internal data. The other app can request this summary using `fetch_summary()` from `integration.py`, which validates the service URL and response before returning a normalized summary.

6. Aura can control a specific focus session. For this flow, Aura creates a temporary focus session and links it to a timer page. The page reads the session ID, token, and API address from the URL fragment, then keeps them in `sessionStorage`. It uses the token to make authorized requests to `/api/focus-sessions/<session_id>`, allowing the user to start, pause, resume, complete, or cancel that particular session.

This is a limited-access design: the token applies to one session rather than granting broad access to the Aura app.

7. Several checks help protect the integration. Timer requests are restricted to the same origin, integration endpoints require tokens, and direct app-to-app calls are limited to loopback addresses. Session-specific tokens are stored in `sessionStorage` instead of being placed in query strings.

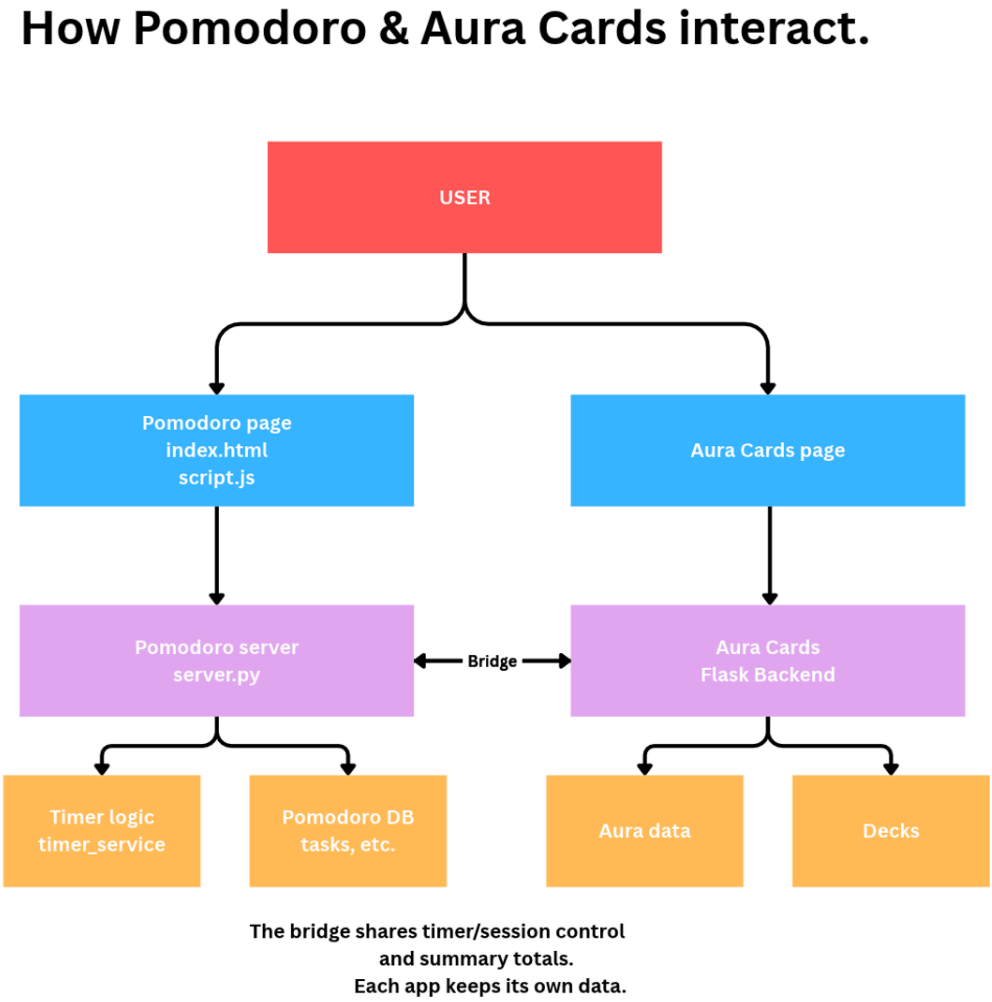


Fig 4: How Pomodoro and Aura cards interact

Users, Inputs (types, formats), Output
Users

- The student and site owner, Pranav Anand
- The course instructor or evaluator, who reviews coursework and project evidence
- Visitors, classmates, and reviewers, who browse the portfolio and its media
- Potential recruiters and other external viewers, who explore the project, live demo, and GitHub links

Inputs

The site's content comes primarily from existing files and links:

- Images (.jpeg): Homework, assessment marks, Figma work, quiz screenshots, accessibility experiments, and a voice-recording photo
- Audio (.mp3, .auc): A project voice note or recording
- Video (.mp4): A project demo
- PDF (.pdf): The Python project writeup
- External links: The live project, Google Drive video, slides, GitHub repository, and writeup document
- Visitor interactions: Clicking navigation links, opening images, playing audio or video, and scrolling through the page

Outputs

The project presents a polished, visually styled portfolio of the student's Python coursework and project work. Visitors can browse a media gallery, view academic and project evidence, play embedded audio and video, read the PDF writeup, and follow links to the demo, GitHub, slides, and other project materials.

References

1. pomofocus.io
2. https://github.com/pranavanand17/python-portfolio/blob/17173dbdae757d03e124ceefaa63da69f7da41e6/index.html
3. https://github.com/pranavanand17/python-portfolio/blob/17173dbdae757d03e124ceefaa63da69f7da41e6/styles.css
4. https://github.com/pranavanand17/python-portfolio/tree/main/Content